

Tracciatura dei tubi a fontana

Modulo Traccia — logica e algoritmi del software legacy

dott. ing. Gian-Lucva ANFOSSI

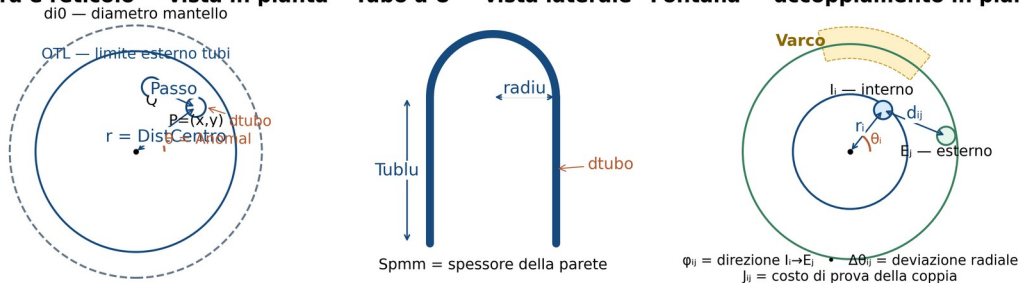
7 ottobre 2026

Table of Contents

Nomenclatura e simboli del software

Questa sezione raccoglie i simboli richiamati nel documento mantenendo, fra parentesi o in codice, lo stesso identificatore usato da **Traccia** e dalle classi collegate di **RoutBase1** e **Infila**. Le grandezze geometriche sono espresse normalmente in millimetri e gradi nell'interfaccia; le routine trigonometriche interne lavorano in radianti.

Piastra e reticolo — vista in pianta **Tubo a U — vista laterale** **Fontana — accoppiamento in pianta**



Significato geometrico dei simboli principali

Nella vista della piastra $di0$ è il diametro disponibile del mantello, OTL è il diametro limite dei tubi, $dtubo$ è il diametro del singolo foro/tubo e $Passo$ è la distanza centro-centro. Il punto $P=(x,y)$ è individuato anche da $r=DistCentro$ e $\theta=Anomal$. Nella U, $radiu$ descrive la curva fisica; nella fontana I_i ed E_j sono gli estremi della coppia e d_{ij} la loro distanza in pianta. Sono quindi grandezze diverse anche quando il legacy usa localmente il nome generico **Raggio**.

Coordinate, diametri e contorni

Simbolo software	Significato operativo	Precisazione
x, y	coordinate cartesiane del centro di un foro	Origine nel centro della piastra; non indicano la posizione nella matrice bu.
PuntiD(i,z)	punto geometrico del foro i nella zona z	Oggetto clsVec2 di RoutBase1; z=0 esterna, z=1 interna per la fontana.
r , Raggio, DistCentro(i,z)	distanza radiale $\sqrt{x^2+y^2}$	Usata per scegliere, bilanciare e ordinare; non è il raggio di curvatura del tubo.
θ , Anomal(i,z)	angolo polare del foro	Calcolato da $\arccos(x/r, y/r)$ e normalizzato nel riordino.
$di0$	diametro imposto o di ingresso del fascio	Il suo ruolo preciso dipende dalla modalità di calcolo bloccata su tubi o diametro.
$di1$	diametro utile/calcolato durante l'iterazione	Viene riconciliato con OTL, giochi e numero tubi.
OTL	<i>Outer Tube Limit</i>	Diametro del minimo contorno circolare che comprende i tubi, includendo mezzo diametro tubo.
otimp	OTL imposto	Vincolo dell'utente distinto dall'OTL calcolato.
otlmax	massimo OTL ammesso/individuato	Limite usato nelle iterazioni geometriche.
dtubo, Diametro	diametro esterno del tubo	Diametro è il nome usato anche nello stato Franco di Infila.
Spmm, Spbwg	spessore tubo in mm e designazione BWG	Rappresentazioni alternative dello stesso dato tecnologico.
Tublu	lunghezza del tubo	Lunghezza rettilinea di progetto, distinta dalla lunghezza della proiezione U.
cinter	raggio/limite della zona centrale libera	VerifInters impedisce al segmento di attraversare questa zona quando significativa.
cori, coriint, coriext	diametri di separazione, interno ed esterno delle corone	Usati esclusivamente nella geometria a fontana.

Reticolo, file, settori e circuitazione

Simbolo software	Significato operativo	Precisazione
Passo	passo nominale del reticolo	Distanza caratteristica fra centri tubo.
PassoOrizzontale, PassoVerticale	componenti cartesiane del passo	Derivano da tipo/orientamento del reticolo.
passoint	passo della corona interna	Può differire dal passo della corona esterna nella fontana.
TipoPasso	codice dello schema del reticolo	Seleziona reticolo triangolare/quadrato e relativo orientamento.
PassoFascio	codice della circuitazione	Determina numero di passaggi, settori e setti.
NumeroSettori	numero dei settori geometrici	Non coincide necessariamente con il numero delle file.
k, Settore(i,z)	indice/settore di appartenenza	k è anche variabile di ciclo legacy; va interpretata dal contesto.
j, Fila(i,z)	indice/fila di appartenenza	Nella parte di matching j è anche usato convenzionalmente per il foro esterno.
nnn, Posizbu(i,z)	posizione nella fila e nella matrice bu	È distinta dall'identificativo enumerato del foro.
bu(fila,posizione)	matrice compatta del layout	"1" foro presente, "0" vuoto, "9" cambio/discontinuità di simmetria.
ici(fila)	numero di posizioni da esaminare nella fila	Limite del ciclo che ricostruisce le coordinate.
hsym, icontr	codici di simmetria e conteggio file del settore	Conservano la rappresentazione compatta legacy.
FilaIniziale, FilaFinale	estremi delle file appartenenti a un settore	Definiscono l'intervallo percorso da CoordInMem.
NumeroTubiFila, NumeroTubiSettore	contatori locali dei fori/tubi	Aggiornati anche dal bilanciamento.
ntubi	numero richiesto di tubi/fori secondo il contesto	Nei fasci U distinguere sempre tubo fisico, gambe e fori.
ktotal	totale effettivo delle posizioni presenti	È il totale enumerato dopo generazione/bilanciamento.
Ntot	contatore totale transitorio del generatore	Usato mentre si formano settori e file.
pdiaf, p1, tagli, tcava	passo diaframmi, parametri dei setti/tagli e cava	Condizionano zone escluse e geometria dei diaframmi.
yin, yout, dtin, dtout	limiti/dimensioni delle zone di ingresso e uscita	Usati per riservare le finestre di passaggio del fluido.

Tubi a U e fontana

Simbolo software	Significato operativo	Precisazione
radiu	raggio minimo/richiesto della curva U	Non confondere con DistCentro.
CurveInPianoVert	orientamento delle curve U nel piano verticale	Modifica simmetrie, settori e conteggi.
Passi4CurveVert	opzione legacy per curve verticali e quattro passaggi	Attiva una gestione specifica della circuitazione.
GapCurve	gioco richiesto fra le curve della fontana	In Infilata viene sommato al diametro per il controllo tridimensionale.
Interf (ε)	interferenza ammessa al montaggio	La distanza planare minima usata da Sorvolo è dtubo-Interf.
Varco	settore senza pause/ostacoli RX	Espresso in gradi; influenza il riordino finale.
Anomal0 (θ ₀)	angolo medio/iniziale del varco	Origine angolare corretta della sequenza finale.
SovrAlt	sovraltezza incrementale	Passo/valore iniziale usato nella simulazione di infilaggio.
AltMin (MINALTO)	tratto diritto minimo in altezza	Vincolo geometrico di Infilata.
Preciso (PRECISO)	precisione della ricerca della distanza minima	Limitata dal codice fra Diametro/100 e Diametro/10.
Hmax, HCalc, Alt, Alt1	altezze correnti e massime di montaggio	Stato iterativo del confronto fra due forcine.
Raggio, Raggio1 in Franco	raggi delle due forcine confrontate	Contesto Infilata, non raggio polare del foro.
Utub, Utub1	proiezioni planari delle due forcine	Oggetti clsLinea2 di RoutBase1.

Simbolo software	Significato operativo	Precisazione
CC, CC1	coseni/direzioni unitarie delle proiezioni	Oggetti clsVec2, usati in proiezioni e distanze.
Lun, Lun1	lunghezze delle proiezioni planari	Limiti dei parametri lungo i segmenti.

Accoppiamento della fontana

Simbolo software	Significato operativo	Precisazione
$I=\{I_i\}$	insieme dei fori della corona interna	Nel VB: PuntiD(i,1).
$E=\{E_j\}$	insieme dei fori della corona esterna	Nel VB: PuntiD(j,0).
i, iTubo1	identificativo del foro interno corrente	È un indice di enumerazione, non un ordine radiale.
j, iTubo2	identificativo del candidato esterno	Nel codice iTubo è usato anche come indice generico di scansione.
d_{ij} , Distl	distanza euclidea fra I_i ed E_j	Lunghezza della corda in pianta.
φ_{ij} , Ang1	angolo del vettore $I_i \rightarrow E_j$	Confrontato con la direzione radiale dell'interno.
$\Delta\theta_{ij}$, DAng	minima differenza angolare	Riportata nell'intervallo $[0,\pi]$.
Fact (F)	peso della deviazione angolare	Aggancia parte da 2 e può diminuirlo di 0,25.
J_{ij} , NewDist	costo $d_{ij}[1+\text{Fact}\cdot\Delta\theta_{ij}/(2\pi)]$	Stabilisce l'ordine di prova, non l'ottimo globale.
NewInd	traduzione posizione-candidato	Contiene l'identificativo reale dell'esterno associato a NewDist.
Accoppia(i,z)	stato del foro	0 libero; altro valore = numero della coppia assegnata.
IndT1(k), IndT2(k)	estremi interno ed esterno della coppia k	Sono i dati principali scritti nel file .COO.
iCoppia (k)	profondità/coppia corrente	Avanza dopo ogni registrazione valida.
iProfond (p)	profondità di ripresa del backtracking	Memorizza il livello dal quale ricostruire.
Proiez (s)	ascissa della proiezione del foro sulla retta	Il foro è fra gli estremi quando $0 < s < d$.
Dist1 (h)	distanza perpendicolare foro-retta	Fa scattare Sorvolo se $< dtubo\text{-}Interf$.

File e risultati

Simbolo	Contenuto
.TRK	progetto/tracciatura legacy serializzata
.COO	coppie, coordinate, file e posizioni della fontana
.PAS	coordinate espresse nel reticolo/passi
.DGE	dati geometrici generali in formato ASCII
.RES	risultati della simulazione d'infilaggio
.MTO	raggruppamenti e dati per distinta materiali
.RB2 / \$.B2	stato/cache interno della simulazione
.SEQ	sequenza di montaggio
.DXF / DGE	esportazione geometrica verso CAD

Mappa fra maschere di input e simboli

Le figure seguenti riproducono schematicamente i gruppi della maschera **Dati di progetto**. Dentro ogni casella bianca non è mostrato un valore di esempio, ma il simbolo che riceve quel dato nelle routine. La corrispondenza è ricavata dagli array `igr1`, `igr2`, `igr3`, `igr41/igr42`, dalle etichette `nominput(n)=at1(106+n)` e dal trasferimento `dt(n) → typDaTos` in Dati.

Campi comuni — pagina iniziale

Dati iniziali — contorno e processo	
Diametro interno mantello [mm]	di0 = dt(2)
D.I. bocchello ingresso [mm]	dtin = dt(3)
D.I. bocchello uscita [mm]	dtout = dt(4)
Densità ingresso [kg/m ³]	roin = dt(5)
Portata ingresso [kg/h]	win = dt(6)
Densità uscita [kg/m ³]	roout = dt(7)
Portata uscita [kg/h]	wout = dt(8)
Zona libera ingresso / uscita [mm]	y0in=dt(9); y0out=dt(10)

Maschera: contorno, bocchelli e portate

Campo mostrato	Slot maschera	Simbolo effettivo
Diametro interno mantello	dt(2)	di0
D.I. bocchello ingresso/uscita	dt(3), dt(4)	dtin, dtout
Densità ingresso/uscita	dt(5), dt(7)	roin, roout
Portata ingresso/uscita	dt(6), dt(8)	win, wout
Zona libera ingresso/uscita	dt(9), dt(10)	y0in, y0out

Tubi diritti e dati generali

Dati del fascio e dei tubi	
Numero tubi	ntubi = dt(11)
Tipo passo / tracciatura / fascio	TipoPasso=dt(12); PassoFascio=dt(13); TipoFascio=dt(14)
Diametro esterno tubo [mm]	dtubo = dt(15)
Passo tubi [mm]	Passo = dt(16)
Passo diaframmi / primo diaframma [mm]	pdiaf=dt(17); p1=dt(18)
Materiale tubo	matub = dt(19)
Raggio tubi U [mm]	radiu = dt(21)
Passo iterazioni [mm]	IncrementoDiametro = dt(22)
Cava / spessore / BWG / lunghezza	tcava=dt(23); Spmm=dt(24); Spbwg=dt(25); Tublu=dt(26)
Tolleranza / giunto tubo-piastra	TipoTolleranza=dt(27); TipoGiunto=dt(28)

Maschera: *reticolo e dati del tubo*

TipoPasso, PassoFascio, TipoFascio, matub, TipoTolleranza e TipoGiunto sono selezioni codificate; gli altri campi sono numerici. Il simbolo IncrementoDiametro è globale alla routine iterativa e non fa parte della struttura typDaTos.

Tubi a U

Nella stessa maschera il campo **Raggio tubi a U (0=STD)** alimenta radiu = dt(21). Le opzioni **Fila centrale storta** e **Curve su piano verticale** alimentano rispettivamente FilaCentraleStorta e Passi4CurveVert/CurveInPianoVert secondo la pagina e la configurazione.

Diaframmi e dati finali

Diaframmi, contorni e accessori	
Taglio / angolo / tipo diaframma	dt(30); dt(31); dt(32)
Piatto d'urto	EsistePiatto = (dt(33)=1)
Eventuale diametro interno corona [mm]	cinter = dt(34)/2
Sealing strips / rod runners	ISEAL=dt(35); Nrod=dt(36)
Tacche	dt(37)
Distanza file su setto H/V [mm]	DistSuSettoHor=dt(38); DistSuSettoVer=dt(39)
Calcolo OTL minimo/massimo	MassimizzaOTL = (dt(40)>0)

Maschera: *diaframmi e limiti geometrici*

La voce **Eventuale diametro interno corona** viene trasformata in cinter=dt(34)/2: la maschera chiede un diametro, mentre le verifiche geometriche lavorano con il raggio.

Fontana

Parametri specifici — tubi a fontana	
Diametro interno corona esterna [mm]	cinter = dt(34)/2
Diametro esterno corona interna [mm]	coriext = dt(44)
Diametro interno corona interna [mm]	coriint = dt(42)
Passo corona esterna [mm]	Passo = dt(16)
Passo corona interna [mm]	passoint = dt(43)
Interferenza ammessa [mm]	Interf = dt(47)
Gioco nelle chiome [mm]	GapCurve = dt(48)
Varco libero RX [°]	Varco = dt(49)
Anomalia media varco [°]	Anomal0 = dt(50)

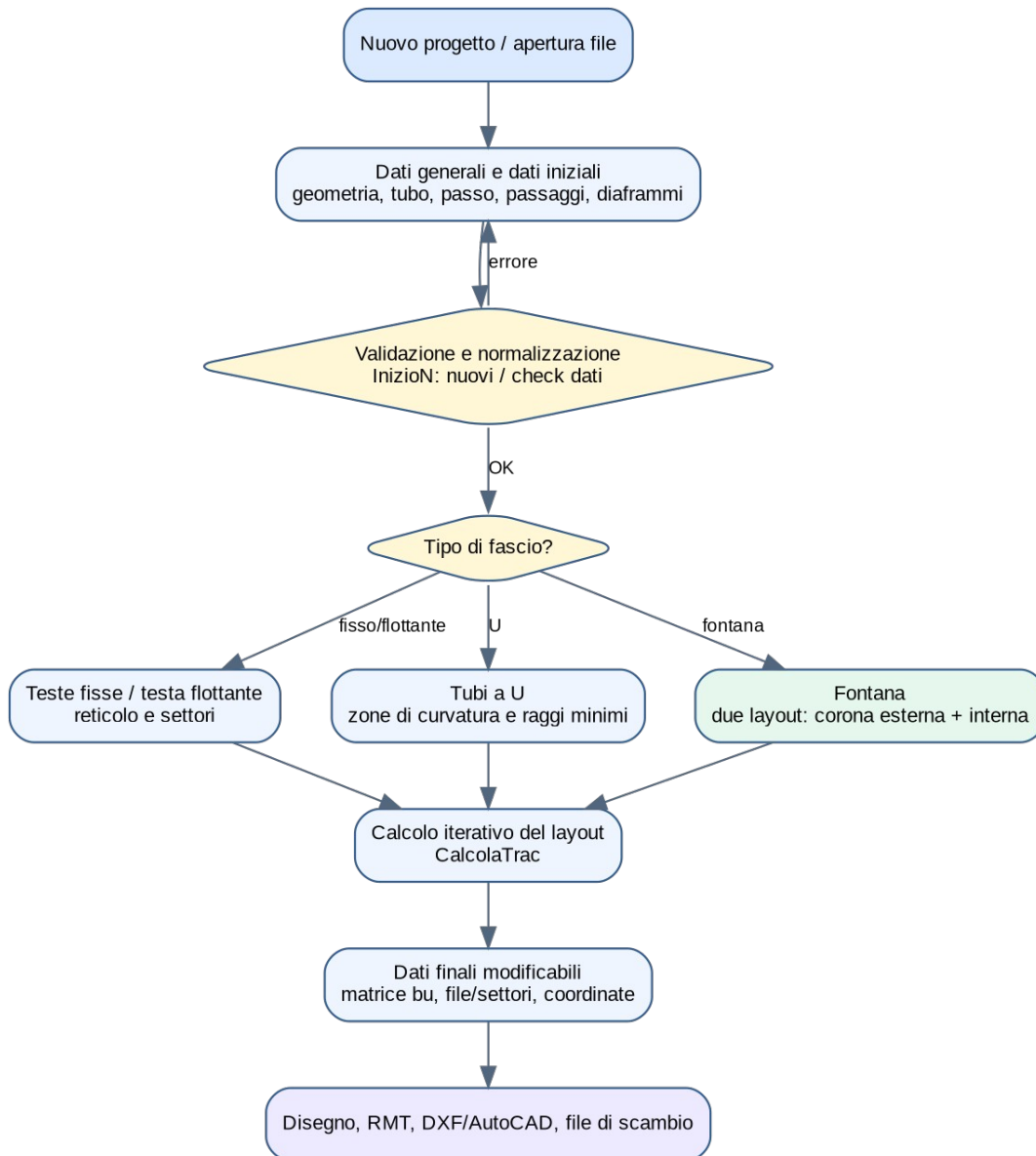
Maschera: parametri specifici della fontana

Campo mostrato	Slot maschera	Simbolo effettivo
Diametro interno corona esterna	dt(34)	cinter=dt(34)/2
Diametro esterno corona interna	dt(44)	coriext
Diametro interno corona interna	dt(42)	coriint
Passo corona esterna	dt(16)	Passo
Passo corona interna	dt(43)	passoint
Interferenza ammessa	dt(47)	Interf
Gioco nelle chiome	dt(48)	GapCurve
Varco libero RX	dt(49)	Varco
Anomalia media del varco	dt(50)	Anomal0

1. Che cosa fa il modulo

Traccia genera la disposizione dei fori sulle piastre tubiere di scambiatori Shell & Tubes. A partire da diametri, tubo, passo, schema del reticolo, numero di passaggi e vincoli geometrici, costruisce file e settori, determina quali posizioni contengono un tubo, calcola OTL e conteggi e produce disegni e file di scambio. Per i fasci a U a fontana aggiunge tre problemi distinti:

1. equilibrare il numero di fori tra corona interna ed esterna;
2. accoppiare ogni foro interno con un foro esterno senza bloccare il montaggio;
3. simulare l'infilaggio tridimensionale e ricavare sovraltezze e sequenza.



Flusso complessivo del modulo

Le strutture principali sono contenute in `clsTracciatura.typDaTos`. La matrice `bu(fila,posizione)` codifica il layout: 1 indica un foro/tubo presente, 0 una posizione vuota e 9 una discontinuità usata per passare al semipiano simmetrico. Gli array `x`, `y`, `NumeroTubiFila`, `FilaIniziale`, `FilaFinale`, `NumeroTubiSettore`, `ici` e `hsym` descrivono la geometria compatta che viene poi trasformata in coordinate.

2. Dati in ingresso, stato e risultati

2.1 Ingressi geometrici essenziali

Gruppo	Variabili legacy indicative	Significato
Contorno	di0, di1, OTL, otimp	diametro imposto/calcolato e outer tube limit
Tubo	dtubo, Spmm, Tublu	diametro, spessore e lunghezza tubo
Reticolo	TipoPasso, Passo, PassoVerticale, PassoOrizzontale	orientamento e passo del reticolo
Circuitazione	PassoFascio, NumeroSettori	numero/configurazione dei passaggi e settori
U-bend	radiu, CurveInPianoVert, Passi4CurveVert	raggio minimo e orientamento curve
Diaframmi	pdiap, p1, tagli, tcava	passo, taglio e cava del setto
Fontana	cori, coriint, coriext, passoint, Interf, GapCurve, Varco	separazione delle corone e vincoli di montaggio

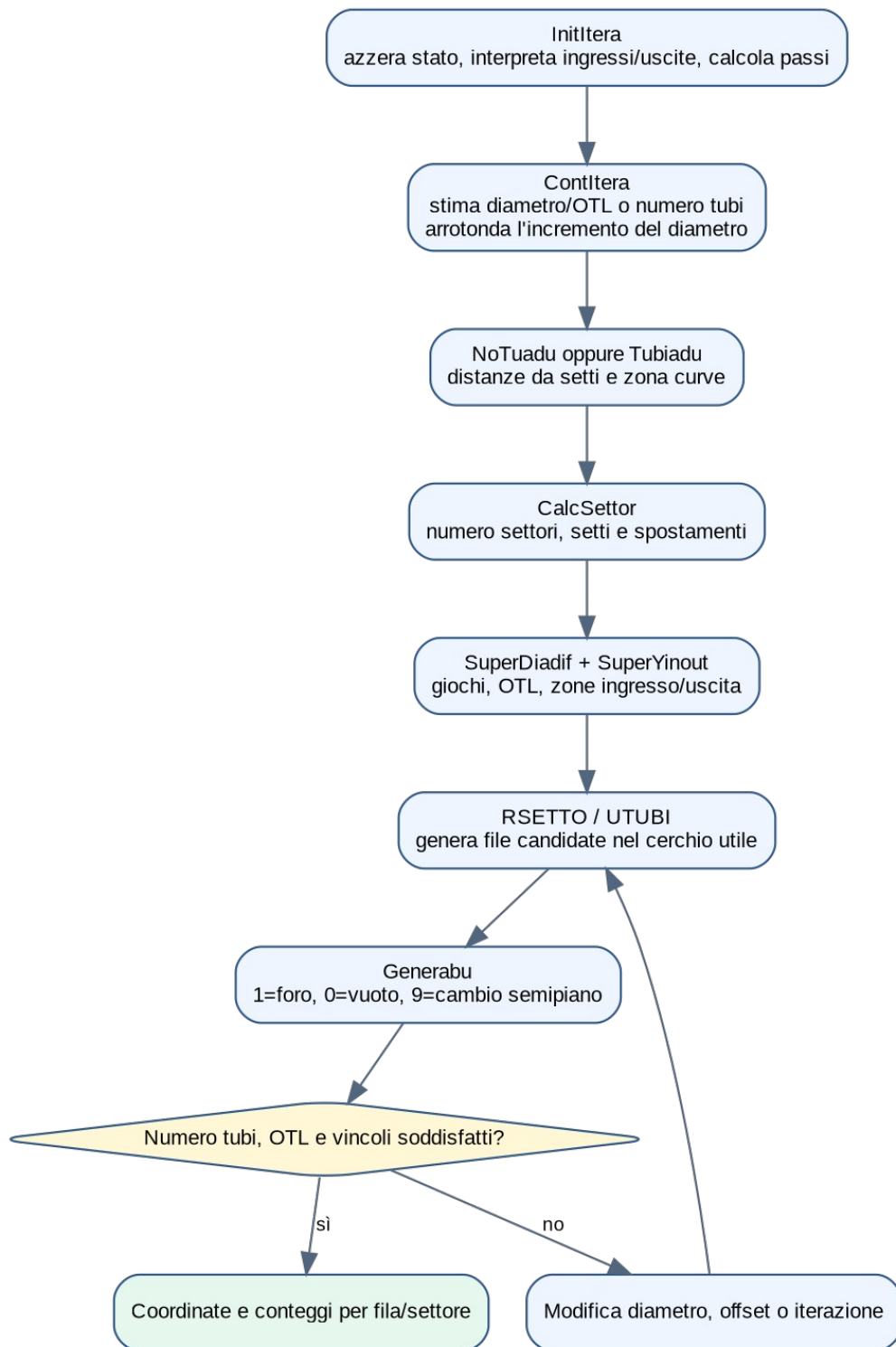
La routine di controllo verifica intervalli ed enumerazioni, coerenza del numero di passaggi, presenza di tubo/passo/diametri e vincoli specifici. Per la fontana il controllo ammette il percorso _2U; per gli U tradizionali sono trattate le configurazioni pari previste dal codice.

2.2 Risultati

- coordinate dei centri dei fori e appartenenza a fila/settore;
- numero totale di tubi/fori e numero per fila e settore;
- OTL, diametro limite, area e perimetro della tracciatura;
- geometria di setti, tagli, tiranti, sealing strips e zone libere;
- per la fontana: coppie interno/esterno, sovraltezze, raggi, ordine di montaggio;
- disegno su schermo, DXF/AutoCAD, RMT e file intermedi/finali.

3. Algoritmo comune di generazione del layout

La routine centrale è **CalcolaTrac**. Non colloca semplicemente punti in un cerchio: esegue un ciclo iterativo perché numero tubi, OTL, diametro e disposizione delle file si influenzano reciprocamente.



Calcolo iterativo del layout

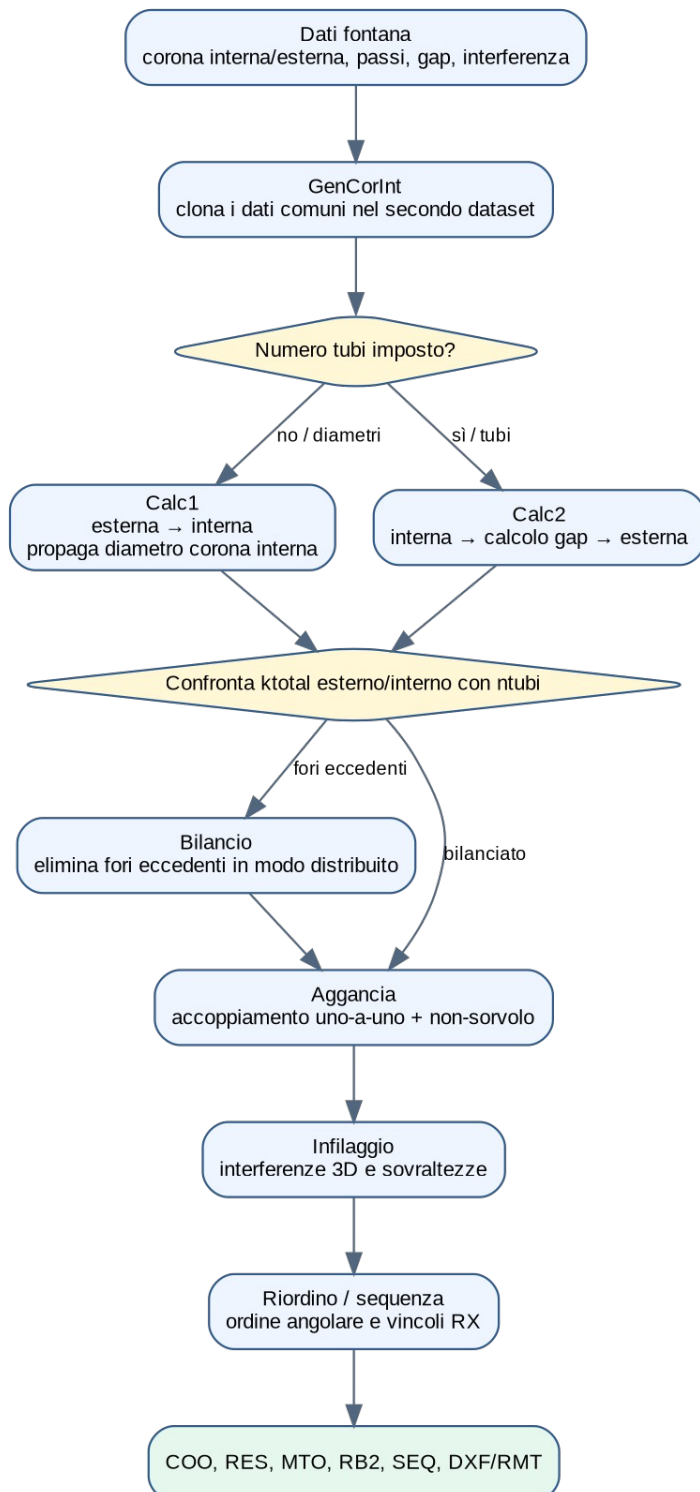
Passo per passo:

1. **InitItera** azzerà lo stato transitorio, interpreta le opzioni di zona ingresso/uscita, stabilisce margini delle cave e calcola le componenti del passo con **RPASSO**.
2. **ContItera** stima un diametro iniziale dall'area del reticolo quando il diametro non è imposto; se manca il numero di tubi ne ricava una stima dall'area utile. Il diametro viene arrotondato a incrementi dipendenti dalla dimensione.
3. **NoTuadu / Tubiadu** definiscono le distanze dai setti; nel caso U trattano la zona di curvatura e il raggio.
4. **CalcSettor** traduce il codice dei passaggi in settori e setti e assegna gli scostamenti orizzontali. Il comportamento cambia per codici pari, dispari e per curve nel piano verticale.
5. **SuperDiadif** calcola il gioco fra fascio e diaframmi; **SuperYinout** uniforma o calcola le zone di ingresso/uscita.
6. **RSETTO / UTUBI** generano le file candidate all'interno del contorno, applicando simmetrie, setti e limiti della curva U.
7. **Generabu** materializza la matrice di presenza. Rimuove le posizioni che cadono nella corona esclusa, aggiorna conteggi e calcola l'OTL massimo.
8. Il ciclo confronta tubi ottenuti, OTL e diametri richiesti; modifica offset o diametro e ripete fino a soluzione o al limite di iterazioni.

4. Tipologie di fascio

4.4 Tubi a U a fontana (Fontana = 4)

La fontana è modellata con **due istanze di typDaTos**: dataset 0 per la corona esterna e dataset 1 per quella interna. Non è quindi un semplice caso grafico del fascio U: richiede due tracciature compatibili e un problema di accoppiamento/montaggio.



Pipeline completa della fontana

5. Fontana: generazione e bilanciamento

5.1 Generazione delle due zone

GenCorInt copia nel dataset interno i dati comuni, re-inizializza gli array e imposta passo e diametro della corona interna. Fontana salva la condizione iniziale e tratta temporaneamente entrambe le zone come layout a teste fisse a un passaggio, così da riutilizzare CalcolaTrac senza duplicarne la matematica.

- Se il numero tubi non è imposto, **Calc1** calcola prima la corona esterna, poi l'interna e propaga il diametro ottenuto.
- Se il numero tubi è imposto, **Calc2** calcola prima l'interna, ricava il gap tra zone e calcola infine l'esterna.

Al termine ripristina il tipo Fontana e _2U. Questa mutazione temporanea è importante per il debug: un breakpoint dentro CalcolaTrac vedrà TipoFascio=TesteFisse, anche se l'operazione complessiva è una fontana.

5.2 Bilanciamento

Le due zone possono produrre numeri diversi di fori e, di norma, non meno del numero richiesto. inizio2 decide lo stato successivo:

- conteggi uguali: passa all'aggancio;
- una zona ha meno fori del richiesto: soluzione impossibile;
- esistono fori eccedenti: richiede bilanciamento.

Bilancio non crea ancora le coppie. Prima DistInMem enumera le posizioni, calcola il raggio e le riordina prevalentemente per raggio decrescente; un secondo passaggio cerca di mantenere vicini settori adiacenti nei gruppi con raggio quasi uguale ($|\Delta r| < 0,1$). Poi elimina dall'inizio dell'elenco tante posizioni quante sono eccedenti, marcando la cella originale bu = "0" e aggiornando i conteggi di fila, settore e corona.

DistInMem() // coordinate + raggio; ordine prevalentemente decrescente

PER zona = ESTERNA, INTERNA

totale = ktotale(zona)

SE ntubi(zona) > 0

obiettivo = ntubi(zona)

ALTRIMENTI SE zona = ESTERNA

obiettivo = ktotale(INTERNA)

ALTRIMENTI

obiettivo = ktotale(INTERNA)

FINE SE

PER i = 1 .. totale-obiettivo

(fila,posizione,settore) = riferimenti del foro ordinato i

bu[filasposizione] = "0"

NumeroTubiFila[filasposizione]--

NumeroTubiSettore[settore]--

ktotale(zona)--

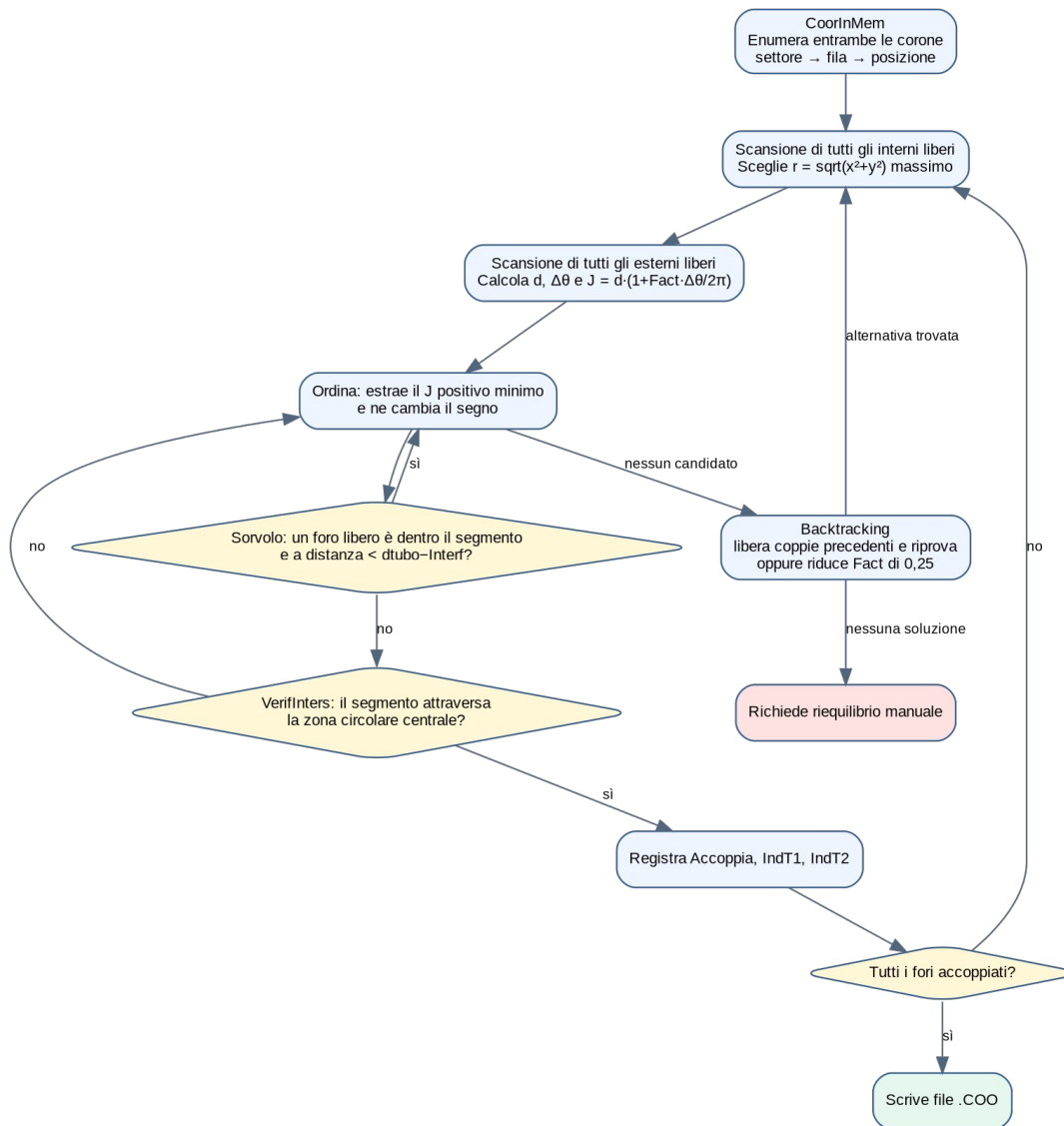
FINE PER

FINE PER

Di conseguenza il bilanciamento modifica il set di fori ammessi **prima** che **CoorInMem** assegni gli identificativi usati da **Aggancia**. Non effettua un matching fra corone e non garantisce da solo che il matching esista; la documentazione legacy prevede infatti correzioni manuali nei casi intrappolati.

6. Fontana: algoritmo di aggancio, in dettaglio

L'aggancio costruisce una corrispondenza biunivoca fra i fori della corona interna (zona = 1) e quelli della corona esterna (zona = 0). Non usa una ricerca globale dell'abbinamento ottimo: è un algoritmo **greedy con filtri e backtracking locale**. Questa distinzione è importante quando si confronta il risultato del programma con un algoritmo moderno di matching.



Algoritmo di aggancio

6.0 Nomenclatura matematica e legacy

Simbolo	Nome	Significato matematico	Nome nel VB legacy
$I=\{I_i\}$	corona interna	insieme dei centri dei fori interni	PuntiD(i,1)
$E=\{E_i\}$	corona esterna	insieme dei centri dei fori esterni	PuntiD(j,0)
i, j	indici	identificativo enumerato del foro interno/esterno	iTubo1, iTubo2/iTubo
(x,y)	coordinate	centro del foro nel piano della piastra	.X, .y
r_i	raggio polare	$\sqrt{x_i^2+y_i^2}$	Raggio, Raggiol, DistCentro
θ_i	angolo polare	$\text{atan2}(y_i, x_i)$, normalizzato	Anomal(i,zona), arco(...)
d_{ij}	distanza	lunghezza della corda $I_i \rightarrow E_j$	Distl

Simbolo	Nome	Significato matematico	Nome nel VB legacy
φ_{ij}	direzione collegamento	angolo del vettore E_j-I_i	Ang1
$\Delta\theta_{ij}$	deviazione radiale	distanza angolare minima tra θ_i e φ_{ij}	DAng
F	peso angolare	bilancia distanza e radialità	Fact, inizialmente 2
J_{ij}	funzione costo	priorità con cui provare E_j per I_i	valore inserito in NewDist
D	diametro tubo	ingombro planare nominale	dtubo
ε	interferenza ammessa	riduce la distanza libera richiesta	Interf
$A(i,z)$	stato assegnazione	0 libero, altrimenti numero coppia	Accoppia(i,z)
$C_k=(i,j)$	coppia k	collegamento interno-esterno registrato	IndT1(k), IndT2(k)
k	indice coppia	profondità corrente della costruzione	iCoppia
p	profondità ritorno	livello da cui riprendere il backtracking	iProfond

La funzione costo corrente è:

$$J_{ij} = d_{ij} \times [1 + F \times \Delta\theta_{ij}/(2\pi)]$$

con:

$$d_{ij} = ||E_j - I_i||_2$$

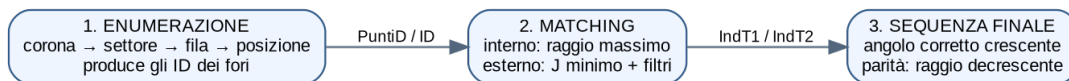
$$\Delta\theta_{ij} = \min(|\varphi_{ij} - \theta_i|, 2\pi - |\varphi_{ij} - \theta_i|).$$

J non è il costo complessivo della configurazione: serve soltanto a stabilire in quale ordine il greedy prova i candidati per l'interno corrente.

6.1 Le tre nozioni di “ordine” non vanno confuse

Il software usa tre ordinamenti differenti, in momenti differenti:

Momento	Criterio effettivo	Scopo
Enumerazione (CoorInMem)	zona, settore crescente, fila crescente, posizione crescente	assegnare un indice stabile a ogni foro
Costruzione delle coppie (Aggancia)	interno libero a raggio massimo; esterni provati per costo crescente J	decidere chi viene collegato con chi
Sequenza finale (Riordino)	angolo corretto con Anomal0; quasi-parità angolare: raggio interno decrescente	stabilire l'ordine di montaggio/controllo



I tre ordinamenti distinti

Quindi la risposta breve è: **il raggio decide quale foro interno trattare per primo; distanza e deviazione angolare classificano gli esterni; l'angolo polare ordina le coppie soltanto dopo che gli abbinamenti sono già stati decisi.**

6.2 Come vengono numerati e memorizzati i fori

CoorInMem percorre separatamente le due corone. Per ciascuna corona visita i settori, le file del settore e le posizioni della fila. CoorXY traduce la matrice compatta bu in coordinate cartesiane; soltanto iCod = 0, cioè una cella bu = "1", diventa un foro disponibile.

PER zona = 0, 1
idForo = 0

```

PER settore = 1 .. NumeroSettori(zona)
  PER fila = FilaIniziale(settore) .. FilaFinale(settore)
    SE fila > 0
      PER posizione = 1 .. ici(fila)
        (x, y, codice) = CoorXY(zona, fila, posizione)
        SE codice = 0
          idForo = idForo + 1
          PuntiD[idForo,zona] = (x,y)
          DistCentro[...] = sqrt(x2+y2)
          Anomal[...] = atan2 normalizzato da arco(...)
          Fila[...] = fila
          Posizbu[...] = posizione
          Settore[...] = settore
        FINE SE
      FINE PER
    FINE SE
  FINE PER
FINE PER

```

L'indice non rappresenta né il raggio né la vicinanza fra le due corone. A parità dei criteri numerici usati più avanti, prevale però il primo elemento incontrato in questo ordine, perché i confronti del legacy usano < o > e non sostituiscono un valore uguale.

6.3 Stato dell'accoppiamento

Struttura	Significato
PuntiD(id,zona)	coordinate cartesiane del centro del foro
Accoppia(id,zona)	0 se libero, altrimenti numero della coppia assegnata
IndT1(coppia)	indice del foro interno della coppia
IndT2(coppia)	indice del foro esterno della coppia
NewInd(k)	indice reale del k-esimo candidato esterno
NewDist(k)	costo del candidato; il segno negativo significa "già provato"
Fila, Posizbu, Settore	provenienza del foro nella matrice geometrica

All'avvio gli array di assegnazione vengono azzerati. Aggancia imposta esplicitamente Fact = 2; pertanto in questa routine il valore iniziale non è quello eventualmente letto in precedenza dall'INI. Se l'intero tentativo deve essere rigenerato, Fact viene diminuito di 0,25, fino al limite descritto nel paragrafo 6.9.

6.4 Scelta del foro della corona interna

Per ogni nuova coppia il programma spazzola **tutti** i fori interni e sceglie quello libero con distanza massima dall'origine:

```

raggioMigliore = -1
interno = nessuno

```

```

PER i = 1 .. numeroForiInterni
  SE Accoppia[i,INTERNA] = 0
    r = sqrt(PuntiD[i,INTERNA].x2 + PuntiD[i,INTERNA].y2)
    SE r > raggioMigliore
      raggioMigliore = r
      interno = i
    FINE SE
  FINE PER

```

```

    FINE SE
  FINE SE
FINE PER

SE interno esiste
  Accoppia[interno,INTERNA] = numeroCoppia
  IndT1[numeroCoppia] = interno
ALTRIMENTI
  accoppiamento terminato
FINE SE

```

È dunque una selezione ripetuta “dall'esterno verso il centro”. Non viene eseguito prima un sort completo: a ogni coppia il massimo viene cercato con una nuova scansione lineare. Due interni allo stesso raggio sono trattati nell'ordine di enumerazione di `CoorInMem`.

6.5 Generazione di tutti i candidati esterni

Fissato l'interno `I`, il codice spazzola l'intero set esterno. Un foro `E` è candidato se non è ancora assegnato. Esiste inoltre il controllo legacy `indiceEsterno <> indiceInterno`: confronta identificativi appartenenti a due array distinti e non una posizione geometrica; va preservato nei test di regressione, ma non va interpretato come filtro di distanza.

Per ogni candidato calcola:

```

d      = sqrt((E.x-I.x)2 + (E.y-I.y)2)
rI     = sqrt(I.x2 + I.y2)
angRad = 0                se rI è circa zero
        arco(I.x/rI, I.y/rI) altrimenti
angIE  = arco((E.x-I.x)/d, (E.y-I.y)/d)
delta  = abs(angIE-angRad)
SE delta > PI: delta = 2*PI-delta

```

$$J(E|I) = d * (1 + \text{Fact} * \text{delta} / (2 * \text{PI}))$$

`delta` è quindi la deviazione minima, compresa fra 0 e π , fra la direzione radiale uscente dall'origine attraverso `I` e il segmento `I→E`. Con `Fact = 2`, la penalità moltiplicativa varia da 1 (direzione radiale) a 2 (direzione opposta). L'algoritmo favorisce collegamenti corti e radiali, ma una maggiore radialità può compensare una distanza geometrica maggiore.

6.6 “Ordina” non ordina l'array: estrae il minimo successivo

La funzione legacy `Ordina` esegue una scansione di `NewDist` e restituisce la posizione del più piccolo valore strettamente positivo. Subito dopo ne cambia il segno. Le chiamate successive ignorano quindi i candidati già provati.

```

FUNZIONE ProssimoCandidato(mode)
  SE mode = RESET
    PER ogni k: SE NewDist[k] < 0: NewDist[k] = -NewDist[k]
  FINE SE

  minimo = +infinito
  posizione = 0
  PER k = 1 .. numeroCandidati
    SE NewDist[k] > 0 E NewDist[k] < minimo

```

```

    minimo = NewDist[k]
    posizione = k
  FINE SE
  FINE PER
  NewDist[posizione] = -NewDist[posizione]
  RITORNA posizione
FINE FUNZIONE

```

È una enumerazione lazy per costo crescente, equivalente a ripetere una selezione del minimo. A parità esatta di costo vince il primo candidato inserito, quindi il primo nell'ordine esterno di **CoorInMem**.

6.7 Primo filtro: Sorvolo

Per la coppia provvisoria $I \rightarrow E$, **Sorvolo** considera **entrambi i dataset** e scandisce tutti i fori ancora liberi, esclusi i due estremi attuali. Per ogni foro libero **P** calcola la proiezione lungo il segmento e la distanza perpendicolare dalla sua retta.

```

L = lunghezza(I,E)
PER zona = ESTERNA, INTERNA
  PER ogni foro libero P della zona, P diverso dall'estremo corrente
    s = proiezione di (P-I) sulla direzione unitaria  $I \rightarrow E$ 
    SE  $0 < s < L$  // P cade fra i due estremi
      h = distanza(P, retta  $I \rightarrow E$ )
      soglia = dtubo(zona) - Interf(zona)
      SE  $h < soglia$ 
        RIFIUTA: la coppia sorvola P
      FINE SE
    FINE SE
  FINE PER
FINE PER
ACCETTA IL FILTRO

```

Il confronto nel sorgente è $h < (dtubo/2)*2 - Interf$, quindi esattamente $dtubo - Interf$. I fori già assegnati non sono controllati qui; lo scopo specifico è non chiudere o ostruire, in proiezione, un foro che deve ancora essere utilizzato. Se il candidato fallisce, viene richiesto a **Ordina** il successivo costo positivo e l'intera scansione ricomincia.

6.8 Secondo filtro: VerifInters

Questo nome può trarre in inganno: nella versione corrente non confronta il segmento con tutte le coppie precedenti. Controlla se $I \rightarrow E$ attraversa la zona circolare centrale, modellata dalla circonferenza di raggio **cinter** con una fascia legata a $dtubo/2$.

```

SE  $cinter < dtubo/2$ 
  RITORNA valido
FINE SE

intersezioni = intersezioni(retta  $E \rightarrow I$ ,
                           cerchio centrato in (0,0),
                           raggio cinter,
                           fascia  $dtubo/2$ )
valido = vero

```

PER ciascuna intersezione Q restituita

a = dot(Q-E, direzione E→I)

b = dot(Q-I, direzione E→I)

SE $a \cdot b < 0$ // Q è internamente al segmento E-I

valido = falso

FINE SE

FINE PER

RITORNA valido

La coppia è ammessa solo se Sorvolo = 0 e VerifInters = vero.

6.9 Cosa succede quando nessun candidato è valido

Esistono due livelli di recupero:

1. **alternativa per lo stesso interno:** finché esistono costi positivi, prova il candidato successivo. Esauriti i candidati, Ordina(mode=1) ripristina i segni e restituisce nuovamente il minimo, che viene fatto transitare nel ramo di fallimento;
2. **backtracking sulle coppie:** il programma arretra nelle coppie già create, cerca una posizione precedente compatibile con l'interno rimasto intrappolato, cancella graficamente e logicamente le assegnazioni dalla profondità scelta in avanti, colloca l'interno nella coppia arretrata e riprende la costruzione.

Pseudocodice strutturale del recupero:

SE candidato fallisce definitivamente

evidenzia interno intrappolato

SE utente annulla: termina senza soluzione

coppiaOriginale = coppiaCorrente

SE $profonditaBacktracking > 0$

coppiaCorrente = $profonditaBacktracking$

RIPETI

coppiaCorrente--

SE coppiaCorrente = 1: termina senza soluzione

esternoPrecedente = IndT2[coppiaCorrente]

SE VerifInters(esternoPrecedente, internoIntrappolato)

E la profondità consente di arretrare

PER c = coppiaCorrente .. coppiaOriginale-1

libera l'interno di c

per c successive libera anche l'esterno di c

cancella il collegamento disegnato

FINE PER

libera l'interno della coppia originale

assegna internoIntrappolato alla coppia arretrata

SE $Sorvolo(internoIntrappolato, esternoPrecedente) = 0$

profonditaBacktracking = coppiaCorrente

riprendi dalla coppia successiva

FINE SE

FINE SE

FINCHÉ non trova un punto di ripresa
FINE SE

Se invece una scansione globale resta incompleta, il codice diminuisce Fact di 0,25, rigenera la tracciatura esterna (SubTraccia(0) e datiout(0)) e ricomincia da zero. Partendo da 2, sono possibili i valori 1,75, 1,50, ..., -0,25; quando la riduzione porta Fact $\leq -0,5$, termina con "Non è stato possibile trovare una soluzione". Un Fact progressivamente più piccolo riduce il peso della direzione angolare rispetto alla distanza.

6.10 Registrazione della coppia e scansione completa

Superati i filtri:

Accoppia[esterno,ESTERNA] = numeroCoppia
IndT2[numeroCoppia] = esterno
disegna segmento interno→esterno
passa alla coppia successiva

Il ciclo termina quando non esiste più un interno libero. Il file .COO registra per ogni coppia: numero, coordinate interne ed esterne, fila e posizione bu dei due estremi e identificativi IndT1/IndT2. Se un .COO preesistente viene accettato dall'utente, il programma recupera direttamente gli identificativi e non riesegue il matching.

6.11 Esempio numerico della graduatoria

Si consideri l'interno $I_1=(30,40)$ mm: il suo raggio è 50 mm e la direzione radiale vale $\theta_1=53,13^\circ$. Tre esterni liberi, tutti a distanza 50 mm, danno con Fact=2:

Candidato	Direzione I→E	$\Delta\theta$	Costo J
A (60,80)	53,13°	0°	50,00
C (30,90)	90°	36,87°	60,24
B (80,40)	0°	53,13°	64,76

L'ordine di prova è A, C, B. Se A sorvola un foro libero, A viene marcato come provato e si passa a C; non si modifica l'interno corrente. Se anche tutti gli altri falliscono, scatta il recupero descritto sopra.

Esempio completo: scelta, scarto, registrazione e ritorno

Supponiamo inoltre $dtubo=20$ mm, $Interf=2$ mm; la distanza minima planare richiesta da Sorvolo è quindi $20-2=18$ mm.

1. **Scelta interno.** Fra gli interni liberi I_1, I_2, I_3 , i raggi sono 50, 42, 30 mm: viene scelto I_1 . Il programma registra provvisoriamente $Accoppia(I_1,1)=1$ e $IndT1(1)=I_1$.
2. **Graduatoria.** $NewInd=[A,C,B]$ nell'ordine di scansione originario e $NewDist$ contiene i rispettivi costi. Ordina trova il minimo A e cambia il suo costo da +50,00 a -50,00.
3. **Scarto A.** Un foro libero P ha proiezione $s=22$ mm, compresa fra 0 e $L=50$ mm, e distanza perpendicolare $h=12$ mm. Poiché $12<18$, Sorvolo restituisce vero: A è scartato.
4. **Prova C.** Ordina ignora il costo negativo di A, seleziona C e lo rende negativo. Nessun foro libero cade nella fascia del segmento e $VerifInters$ non trova attraversamenti della zona centrale: C è valido.
5. **Registrazione.** Il programma scrive $Accoppia(C,0)=1$, $IndT2(1)=C$ e disegna $I_1 \rightarrow C$. Passa quindi a $k=2$.
6. **Nuovo interno.** Il massimo raggio libero è ora $I_2=42$ mm. Ricostruisce da zero la lista dei soli esterni ancora liberi; C non compare più.

7. **Intrappolamento ipotetico.** Se per I_3 tutti gli esterni rimasti falliscono, il codice segnala la coppia corrente, arretra a una coppia precedente e verifica se il suo esterno può essere conservato sostituendo l'interno precedente con I_3 .
8. **Liberazione.** Se arretra alla coppia 1, azzerla gli **Accoppia** degli interni dalla coppia 1 in avanti, azzerla gli esterni delle coppie successive, cancella i relativi **IndT1/IndT2** e le linee disegnate. L'esterno della coppia di innesto può essere temporaneamente conservato.
9. **Configurazione alternativa.** Assegna I_3 a quella coppia, ripete **Sorvolo** sul collegamento conservato e, se valido, ricomincia a scegliere gli interni liberi dal livello seguente. Non genera tutte le permutazioni: esplora soltanto i rami raggiungibili da questa politica di ritorno.

Questo esempio mostra perché una scelta localmente economica può produrre un vicolo cieco: usare C per I_1 potrebbe sottrarre a I_3 il suo unico esterno ammissibile. Il backtracking tenta di correggere la decisione, ma non possiede una funzione euristica globale che riconosca anticipatamente tale scarsità.

6.12 Complessità e limiti osservabili

Con N fori per corona, la scelta ripetuta degli interni è $O(N^2)$. Per ogni interno si generano fino a N candidati; ogni estrazione del minimo è lineare e ogni **Sorvolo** scandisce fino a $2N$ fori. Nel caso sfavorevole il solo matching è quindi dell'ordine di $O(N^3)$, prima del backtracking. Questo spiega parte della lentezza sui fasci grandi.

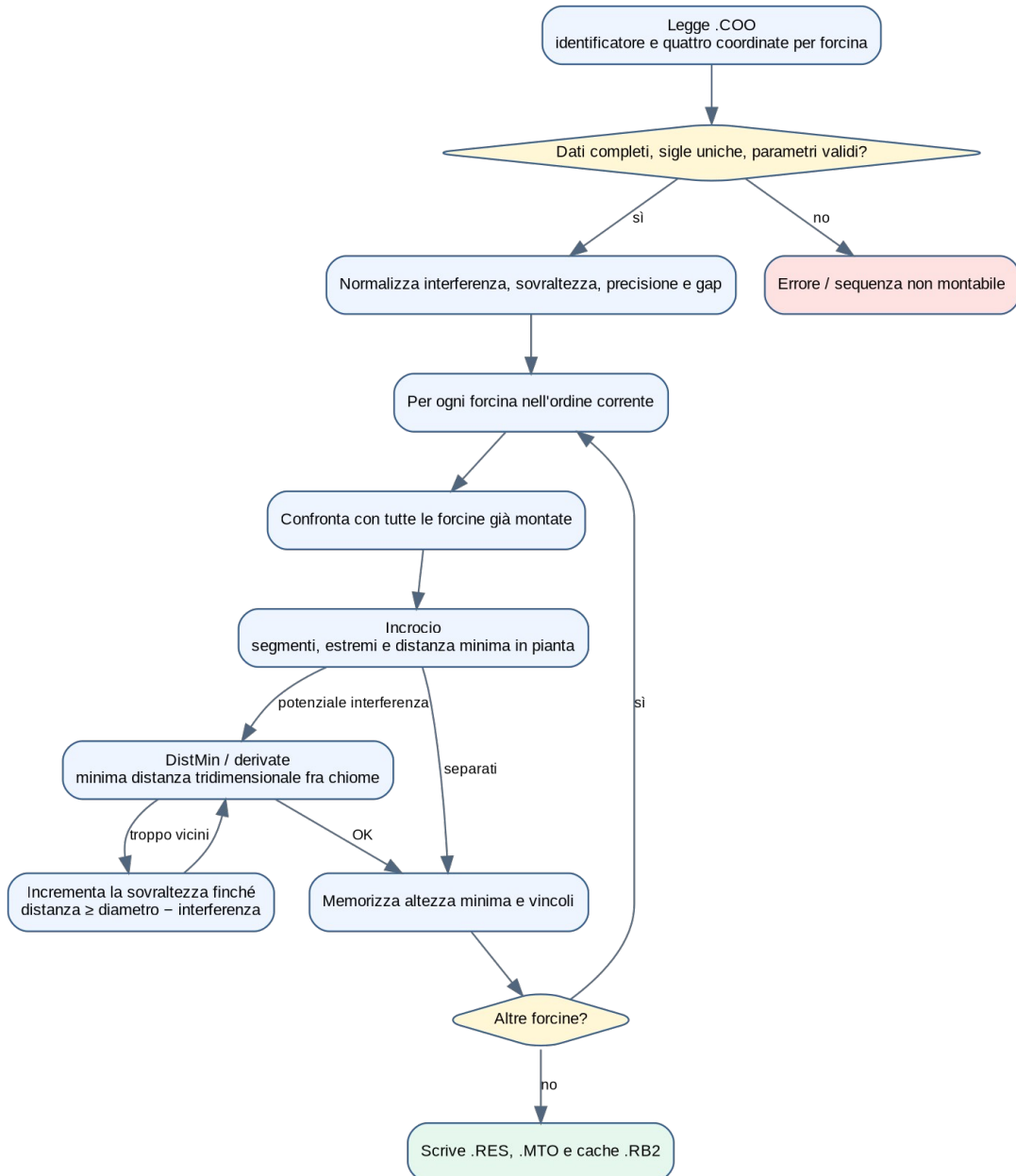
Con il backtracking il limite teorico diventa esponenziale: nel caso peggiore si possono esplorare molte delle permutazioni degli esterni, fino a un ordine di grandezza $O(N!)$. Il legacy non memorizza gli stati già falliti e può ripetere configurazioni equivalenti. Le principali cause concrete di lentezza sono:

- scansioni lineari ripetute per massimo raggio e minimo costo;
- **Sorvolo** che riesamina tutti i fori liberi per ogni candidato;
- assenza di indice spaziale per interrogare soltanto i fori vicini al segmento;
- backtracking senza cache, euristica globale o bound sul costo residuo;
- rigenerazione completa con **SubTraccia** quando cambia **Fact**;
- disegno immediato, **Refresh**, aggiornamento della status bar e **Application.DoEvents** nel ciclo numerico;
- successiva verifica tridimensionale dell'infilaggio, a sua volta iterativa.

Il controllo di non-sorvolo non dimostra da solo la montabilità tridimensionale. Quest'ultima viene affrontata successivamente da **Infila**. Inoltre l'algoritmo greedy non garantisce il matching globalmente minimo: l'ordine di enumerazione e le scelte precedenti possono influire sul risultato.

7. Fontana: algoritmo di infilaggio

L'infilaggio simula l'introduzione successiva delle forcine. Il codice è nel modulo **Infila**; riceve diametro, sovraltezza, interferenza ammessa, altezza minima, precisione e gap delle curve.



Simulazione di infilaggio

7.1 Preparazione

Il programma legge .COO, controlla che ogni identificativo sia unico e che ogni record abbia le quattro coordinate degli estremi. Normalizza i parametri:

- Interf non può superare metà diametro;
- sovraltezza assente → circa $\text{diametro}/10$;
- precisione limitata tra $\text{diametro}/100$ e $\text{diametro}/10$, con default $\text{diametro}/20$;
- il gap di curva interno usa $\text{GapCurve} + \text{diametro}$.

7.2 Controllo geometrico

Per ogni forcina, confrontata con quelle già montate:

- **Incrocio** verifica in pianta intersezioni tra segmenti e distanze degli estremi dalle altre linee usando la soglia **diametro – interferenza**;
- quando la proiezione può interferire, le routine **Dist**, **DerDist**, **DerDerDist** e **DistMin** cercano la distanza minima tridimensionale fra le chiome parametrizzate. La ricerca combina avanzamento limitato e interpolazione sulla derivata;
- se la distanza non è sufficiente, aumenta l'altezza/sovraltezza e ripete;
- memorizza per ciascuna forcina l'altezza minima compatibile e il riferimento all'elemento limitante.

L'output **.MTO** è poi usato anche dalla distinta: contiene gruppi, raggio e lunghezza rettilinea. **.RES** e **.RB2** conservano risultati e stato intermedio.

8. Sequenza, raggi X e output

Riordino ordina le coppie a partire dall'angolo **Anomalo** e, a parità angolare, dalla distanza dal centro.

SorvoloRX confronta i collegamenti già marcati con pausa radiografica: rifiuta intersezioni interne e distanze minori di diametro – **Interf**. Il parametro **Varco** definisce un settore senza pause.

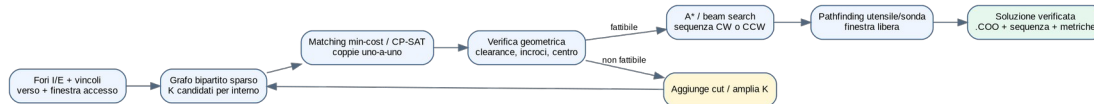
Gli output osservati nel codice comprendono:

Estensione/oggetto	Contenuto o uso
.TRK	progetto/tracciatura serializzata legacy
.COO	coppie e coordinate per la fontana
.RES	risultato dell'infilaggio
.MTO	gruppi di forcine per distinta materiali
.RB2	cache serializzata dello stato d'infilaggio
.SEQ	specifica di sequenza di montaggio
DXF/DGE	geometria per AutoCAD
RTF/Word	relazioni, tabelle e richiesta materiali

9. Come migliorare l'algoritmo

L'evoluzione dovrebbe separare esplicitamente tre problemi oggi intrecciati:

1. **matching**: trovare la migliore corrispondenza biunivoca $I \leftrightarrow E$;
2. **fattibilità geometrica**: escludere sorvoli, zona centrale, incroci e distanze insufficienti;
3. **sequenza di montaggio**: ordinare le forcine in un verso scelto lasciando accessibile il giunto tubo-piastra e le zone da controllare.



Architettura proposta dell'ottimizzatore

9.1 Funzione obiettivo moderna

Per ogni arco candidato (i,j) si può definire un costo normalizzato:

$$\begin{aligned}
 c(i,j) = & wd * d(i,j)/dRif \\
 & + wa * \text{deltaTheta}(i,j)/PI \\
 & + wx * \text{penalitaIncrocio}(i,j) \\
 & + wc * \text{penalitaClearance}(i,j) \\
 & + ww * \text{penalitaFinestra}(i,j) \\
 & + ws * \text{penalitaScarsita}(i,j)
 \end{aligned}$$

dove:

- wd , wa controllano lunghezza e radialità;
- penalitaIncrocio è infinita per una violazione rigida, oppure graduata se si vuole lasciare la verifica tridimensionale alla fase successiva;
- penalitaClearance cresce quando la distanza minima si avvicina a $d_{\text{tubo-Interf}}$;
- penalitaFinestra proibisce o scoraggia segmenti dentro il settore che deve restare accessibile;
- penalitaScarsita protegge un esterno che rappresenta una delle poche opzioni ammissibili per un altro interno.

Prima dell'ottimizzazione conviene costruire un **grafo bipartito sparso**: un nodo per ogni interno, un nodo per ogni esterno e un arco solo se supera i filtri geometrici statici. Per ridurre l'ampiezza della ricerca, per ogni interno si mantengono soltanto, per esempio, i K migliori candidati entro una finestra angolare e di distanza; K deve però poter crescere automaticamente se il grafo non ammette un matching completo.

9.2 Algoritmi candidati e loro ruolo

Metodo	Uso appropriato	Pregi	Limiti
Hungarian / min-cost matching	coppie uno-a-uno con costi indipendenti	ottimo globale polinomiale	non rappresenta bene vincoli fra segmenti
Min-cost max-flow	matching con capacità, classi o penalità aggiuntive	flessibile e ottimo sul grafo	vincoli geometrici fra coppie richiedono iterazioni/cut
CP-SAT / MILP	matching con incroci vietati, finestre e regole logiche	modello dichiarativo, prova di ottimalità o gap	costo elevato sui casi grandi
A*	costruzione con ordine, accessibilità e stato dipendente dalla storia	usa un'euristica e può trovare l'ottimo	memoria elevata; serve euristica ammissibile
Beam search	approssimazione di A* mantenendo solo B stati	memoria e tempi controllabili	non garantisce l'ottimo
DFS + branch-and-bound	evoluzione naturale del backtracking legacy	semplice, riproducibile	può esplodere senza buoni bound

Per il solo matching, A^* non è la prima scelta: Hungarian o min-cost flow risolvono direttamente la corrispondenza. Se però la validità della prossima coppia dipende da quali forcine sono già montate e dalla zona ancora accessibile, lo stato diventa sequenziale e A^* è appropriato.

9.3 Proposta A^* per matching e montaggio congiunti

Uno stato può essere definito come:

$S = (\text{coppieScelte},$
 $\text{interniLiberi},$
 $\text{esterniLiberi},$
 $\text{ordineMontaggio},$
 $\text{frontieraAngolare},$
 $\text{verso},$
 $\text{finestraAccessoResidua})$

Un'azione aggiunge una coppia fattibile alla prossima posizione di montaggio. Il costo accumulato $g(S)$ somma i costi delle coppie e le penalità di montaggio già certe. L'euristica $h(S)$ deve sottostimare il costo residuo; un buon bound è il costo del matching minimo fra i fori rimasti ignorando le interferenze dinamiche. La priorità è $f(S)=g(S)+h(S)$.

OPEN = coda con priorità contenente stato iniziale

CLOSED = mappa stato $\rightarrow$ miglior g noto

MENTRE OPEN non è vuota

$S =$ estrai stato con f minimo

SE S è completo: ritorna soluzione

SE esiste in CLOSED un costo $\leq g(S)$: continua

CLOSED[S] = $g(S)$

$i =$ scegli interno più vincolato

// minimo numero di esterni ancora ammissibili, non raggio massimo

PER ogni esterno j candidato, ordinato per costo incrementale

SE viola matching uno-a-uno: scarta

SE viola Sorvolo/clearance/zona centrale: scarta

SE incrocia una coppia incompatibile: scarta

SE chiude la finestra di accesso prima del controllo: scarta

SE rende un altro interno privo di candidati: scarta

$S2 =$ applica coppia (i,j) e aggiorna frontiera/accessibilità

$g(S2) = g(S) + \text{costoIncrementale}$

$h(S2) = \text{lowerBoundMatching}(\text{fori rimasti})$

inserisci $S2$ in OPEN

FINE PER

FINE MENTRE

ritorna nessuna soluzione

La scelta "interno più vincolato" è la strategia MRV (*minimum remaining values*): tratta prima il foro con meno alternative. È spesso più efficace del raggio massimo nel prevenire gli intrappolamenti osservati nel legacy.

9.4 Montaggio orario o antiorario

Si introduce un angolo di riferimento θ_0 , normalmente al centro del varco, e una coordinata angolare orientata:

$$\alpha_{CW}(i) = \text{mod}(\theta_0 - \theta_i, 2\pi)$$

$$\alpha_{CCW}(i) = \text{mod}(\theta_i - \theta_0, 2\pi)$$

L'utente seleziona verso=CW oppure CCW. La sequenza deve rispettare una frontiera monotona, salvo una tolleranza o un numero massimo di inversioni:

SE verso = CW: $\alpha_{CW}(\text{nuova}) \geq \text{frontiera} - \text{tolleranza}$

SE verso = CCW: $\alpha_{CCW}(\text{nuova}) \geq \text{frontiera} - \text{tolleranza}$

È utile calcolare entrambe le soluzioni e confrontare costo, altezza massima, numero di pause e qualità dell'accesso. Il verso non dovrebbe cambiare le coppie se il matching e la sequenza sono risolti separatamente; può invece influenzarle nell'ottimizzazione congiunta.

9.5 Finestra libera per accesso e controllo del giunto

La finestra deve essere un vincolo geometrico esplicito, non soltanto una preferenza grafica. Si definiscono:

- settore angolare $[\theta_{\text{start}}, \theta_{\text{end}}]$;
- raggio minimo/massimo della zona operativa;
- larghezza utensile o sonda più margine;
- fase fino alla quale la finestra deve restare libera;
- eventuali corridoi radiali di ingresso e uscita.

Una coppia è vietata durante la fase protetta se la sua **capsula geometrica** (segmento dilatato del raggio di ingombro) interseca il settore/corridoio. Questo è più sicuro che controllare soltanto la linea centrale.

funzione Accessibile(coppieMontate, finestra, utensile):

ostacoli = capsule(coppieMontate, raggioIngombro)

corridoio = settoreFinestra eroso del margine utensile

ritorna esistePercorsoLibero(ingresso, giuntiDaControllare,
corridoio - ostacoli)

esistePercorsoLibero può usare A* su una griglia 2D/3D o, meglio, su un grafo di visibilità/configuration space. Qui il pathfinding è applicato al percorso della sonda/utensile, non al matching astratto dei fori.

Si possono imporre due modalità:

1. **finestra permanente:** nessuna forcina può invaderla;
2. **finestra temporanea:** resta libera fino al controllo, poi le ultime forcine possono chiuderla. Queste vengono marcate come gruppo finale.

9.6 Riduzione della ricerca senza perdere soluzioni importanti

Filtri consigliati, nell'ordine:

1. limiti radiali e angolari economici;
2. **Sorvolo** con indice spaziale invece di scansione totale;
3. verifica zona centrale;
4. controllo che ogni nodo rimasto abbia almeno un arco;
5. test rapido di esistenza di matching completo;
6. interferenze segmento-segmento tramite R-tree/k-d tree;
7. verifica tridimensionale soltanto sui candidati sopravvissuti.

La lista dei candidati può partire con $K=8$ o con una semiampiezza angolare limitata. Se non esiste un matching completo, si amplia progressivamente K /finestra; in questo modo la riduzione non diventa un rifiuto definitivo arbitrario.

9.7 Complessità prevista della soluzione moderna

- costruire ingenuamente tutti i costi richiede $O(N^2)$; un k-d tree limita la ricerca dei vicini, ma i casi densi restano quadratici;
- Hungarian è $O(N^3)$ con memoria $O(N^2)$;
- min-cost flow dipende da vertici, archi e implementazione, ma sul grafo sparso con K candidati usa circa $O(NK)$ archi;
- CP-SAT/MILP e A* hanno complessità esponenziale nel caso peggiore;
- A* consuma memoria proporzionale agli stati aperti; beam search la limita a B stati per livello, al prezzo della garanzia di ottimalità;
- i controlli geometrici passano da scansioni $O(N)$ per candidato a query circa $O(\log N + \text{risultati})$ usando un indice spaziale.

La strategia pratica suggerita è: matching min-cost sul grafo sparso, verifica geometrica, aggiunta iterativa di *cut* per combinazioni incompatibili, quindi beam search/A* per la sequenza e la finestra di accesso. Per casi piccoli o di certificazione, CP-SAT può fungere da soluzione di riferimento.

9.8 Migrazione sicura e validazione

Il nuovo algoritmo non deve sostituire subito il legacy. Va introdotto dietro un'interfaccia comune e confrontato su commesse reali:

PairingResult Solve(Layout interno, Layout esterno, Constraints, Strategy)

Per ogni caso si devono conservare: .COO legacy, soluzione nuova, costo, clearance minima, attraversamenti, altezza massima, ordine, verso, finestra libera e tempo di calcolo. I golden test devono verificare prima la fattibilità; l'ottimalità viene valutata soltanto dopo aver dimostrato che non sono cambiate le convenzioni geometriche.

10. Percorso consigliato nel debugger

Fontana

1. breakpoint in `InizioN.Fontana` e `GenCorInt`;
2. seguire `Calc1` o `Calc2` e alternanza di `QualeCorona`;
3. osservare `inizio2` per la decisione bilanciamento/aggancio;
4. in `Bilancio`, controllare quali celle passano da 1 a 0;
5. in `Aggancia`, monitorare `iCoppia`, `iTubo1`, `NewDist`, `iTubo2`, `Accoppia`, `Fact` e i ritorni di `Sorvolo`;
6. in `Infila.Infilaggio`, fermarsi su `Incrocio` e `DistMin`, osservando altezza, distanza minima e forcina limitante;
7. verificare `.COO`, `.RES`, `.MTO` e `.SEQ` prodotti nella commessa.

11. Mappa del codice

File	Responsabilità principale
Tracciatura.vb	API del modulo, struttura dati, lettura/scrittura, disegno e documenti
InizioN.vb	generatore geometrico, iterazione, fontana, bilancio, aggancio e sequenza
Infila.vb	simulazione di montaggio e distanza tridimensionale tra forcine
frmTracciat.vb	interazione utente, disegno e modifica dei dati finali
Tr13N.vb	inizializzazione, caricamento e collegamento con Lancio/PPSM
OpFin.vb, frmDatiOut.vb	opzioni e presentazione degli output

12. Rischi tecnici da preservare nella futura riscrittura

- distinguere sempre **numero tubi**, **numero fori** e **numero gambe**;
- preservare convenzioni $bu=0/1/9$, indici base 1 e simmetrie $hsym$;
- rendere esplicita la mutazione temporanea Fontana → TesteFisse;
- isolare la funzione obiettivo dell'aggancio e renderne configurabile $\bar{F}act$;
- trasformare backtracking, criteri di arresto e messaggi in risultati tipizzati;
- sostituire gradualmente file binari/cache senza perdere la lettura storica;
- costruire golden tests su layout reali prima di estrarre il core in F#.

Appendice A — Fonti analizzate

La descrizione è ricavata dai sorgenti correnti `src/Traccia/*.vb` e dalla documentazione originale conservata in `legacy/traccia.NET/Aiuto`, in particolare `Panoramica.htm`, `Procedura.htm`, `Prototipi.htm`, `fontGen.htm`, `fontBil.htm` e `fontAgg.htm`. Dove la documentazione legacy è incompleta, il comportamento è descritto direttamente dalle routine eseguibili.